

Guía de Consulta Práctica

Trabajo Práctico 02 - Paradigma Orientado a Objetos

El Paradigma Orientado a objetos (POO) es una forma de pensar, diseñar y construir sistemas basada en la idea de que el mundo puede representarse como un conjunto de **objetos** que interactúan entre sí. Cada objeto tiene una **identidad**, **estado** y **comportamiento**, y se relaciona con otros para cumplir funciones dentro de un sistema.

Este enfoque permite construir sistemas más flexibles, reutilizables y cercanos a la forma en que pensamos y resolvemos problemas en la vida cotidiana.

Características Fundamentales del POO

Concepto	Descripción	Ejemplo
Abstracción	Representar sólo los aspectos relevantes de un objeto, ocultando los detalles innecesarios.	Un "Pedido" puede tener fecha, productos y estado, sin mostrar cómo se calcula el total.
Encapsulamiento	Agrupar datos y comportamientos en una misma unidad (la clase), protegiendo su acceso desde fuera.	La clase "CuentaBancaria" tiene un método depositar() que modifica el saldo sin exponerlo directamente.
Ocultamiento de información	Restringir el acceso directo a los datos internos del objeto, permitiendo solo operaciones controladas.	El saldo de una cuenta no puede modificarse directamente, solo mediante métodos como retirar().
Herencia	Permitir que una clase derive de otra, heredando atributos y comportamientos.	"Empleado" puede ser una clase base, y "EmpleadoPorHora" una subclase que agrega cálculo de salario.
Polimorfismo	Una misma operación puede aplicarse a distintos objetos, y cada uno responde de forma diferente.	procesar() puede significar algo distinto para un "Pedido", una "Reserva" o una "Solicitud".

Identidad, Estado y Comportamiento de los Objetos

Todo objeto en el paradigma orientado a objetos se caracteriza por tres aspectos fundamentales:

Aspecto	Descripción	Ejemplo aplicado
Identidad	Es lo que permite distinguir un objeto de otro, incluso si tienen el mismo estado. Cada objeto tiene una existencia única.	Dos objetos "Cliente" con el mismo nombre y DNI siguen siendo distintos si fueron creados en momentos diferentes o tienen trayectorias distintas en el sistema.
Estado	Representa los valores actuales de los atributos del objeto. El estado puede cambiar con el tiempo.	Un objeto "Pedido" puede tener estado = "pendiente", luego "procesado", luego "entregado".
Comportamiento	Son las acciones que el objeto puede realizar, definidas por sus métodos.	El objeto "Pedido" puede ejecutar calcularTotal(), confirmar(), cancelar() según el contexto.

Identificar Objetos en un Escenario

La identificación de objetos en un escenario es una etapa clave en el análisis orientado a objetos, ya que permite construir un modelo conceptual que refleje con precisión las entidades, procesos y relaciones del sistema.

Este proceso no se limita a extraer sustantivos de una narrativa, aunque el método de frases nominales es una estrategia inicial útil: al subrayar los sustantivos en la descripción funcional, es posible detectar candidatos a clases como cliente, reserva, habitación o sistema de pago.

Sin embargo, como advierte Sommerville (2005), el modelado efectivo requiere ir más allá de la superficie lingüística y comprender profundamente los requisitos del sistema.

Esto implica distinguir entre entidades que deben mantenerse en el sistema (por persistencia o trazabilidad), aquellas que cambian de estado, y las que representan reglas, decisiones o políticas.

Craig Larman (2005) propone una clasificación conceptual que ayuda a diversificar el tipo de objetos modelados.

Según su enfoque, los objetos pueden agruparse en cinco categorías:

- Artefactos Físicos (como productos o documentos).
- Roles o Personas (cliente, empleado).
- Eventos o Procesos (reserva, devolución).
- Interfaces Externas (sistemas de pago, bases de datos).
- Conceptos Abstractos (estado, política, solicitud).

Esta taxonomía permite pensar más allá de lo visible y modelar entidades que organizan la lógica del sistema, como validadores, gestores o controladores.

Sommerville complementa esta visión al destacar la importancia de la abstracción: modelar lo esencial y omitir lo accesorio, evitando clases técnicas como “BaseDeDatos” o “Formulario” si no aportan al modelo conceptual.

Para facilitar este proceso, es útil formular preguntas guía que orienten la reflexión:

- ¿Qué entidades participan en la actividad?
- ¿Qué información necesita cada una para cumplir su rol?
- ¿Qué acciones realiza?
- ¿Qué elementos cambian de estado?
- ¿Qué objetos generan o reciben mensajes?
- ¿Cuáles deben persistir en el sistema?
- ¿Qué entidades podrían generalizarse o especializarse?

Estas preguntas permiten validar la relevancia de cada clase y su coherencia con los principios del paradigma orientado a objetos, como encapsulamiento, responsabilidad y colaboración.

En síntesis, identificar objetos no es solo una tarea técnica, sino una práctica de análisis conceptual que requiere observar el sistema desde múltiples perspectivas: lingüística, funcional, estructural y abstracta.

Integrar las estrategias de Larman y Sommerville permite construir modelos más ricos, flexibles y fieles al dominio del problema.

Estrategias para Identificar Objetos en un Escenario

1. Método de Frases Nominales

Esta técnica consiste en **leer atentamente la narrativa del sistema** (escenario, caso de uso, simulación) y **subrayar los sustantivos**. Muchos de ellos pueden convertirse en clases si cumplen alguno de los siguientes criterios:

- Representan una **entidad con información propia** (estado).
- Participan activamente en el sistema (comportamiento).
- Interactúan con otros elementos (relaciones).
- Cambian de estado o responden a eventos.

Ejemplo aplicado: En el escenario “Un cliente realiza una reserva en un hotel”, los sustantivos relevantes podrían ser: cliente, reserva, habitación, disponibilidad, sistema, pago. Cada uno puede evaluarse como clase potencial si tiene atributos y responsabilidades propias.

2.- Lista de Categorías

Esta estrategia ayuda a diversificar el tipo de clases y evitar que el modelo se limite únicamente a entidades físicas o visibles.

En el análisis orientado a objetos, es común que los estudiantes tiendan a identificar como clases sólo los elementos tangibles del sistema (como productos, clientes o documentos).

Sin embargo, para construir un modelo conceptual completo, es necesario incluir también objetos que representen **roles funcionales, procesos, interfaces y conceptos abstractos** que organizan la lógica del sistema.

Craig Larman propone una clasificación útil para ampliar esta mirada. Según su enfoque, los objetos pueden agruparse en las siguientes categorías:

Categoría	Ejemplos	Aplicación
Artefactos físicos	producto, documento, ticket	Representan objetos tangibles que circulan o se gestionan.
Roles o personas	cliente, empleado, proveedor	Representan actores que interactúan con el sistema.
Eventos o procesos	reserva, compra, devolución	Representan acciones o flujos que deben modelarse como objetos.
Interfaces externas	sistema de pago, Sistema de inventario	Representan sistemas con los que se comunica el sistema modelado.
Conceptos abstractos	solicitud, estado, política	Representan entidades lógicas que organizan la información o el comportamiento.

La Importancia de Modelar Conceptos Lógicos y Reglas del Sistema como Objetos

En el paradigma orientado a objetos, no todos los objetos deben ser físicos o visibles.

Algunos de los más importantes son aquellos que organizan el comportamiento del sistema, definen reglas, estados, condiciones o políticas.

Estos objetos no existen como entidades tangibles, pero son fundamentales para que el sistema funcione correctamente y para que el modelo refleje su complejidad real.

Por ejemplo:

- **EstadoReserva** puede representar los distintos estados posibles de una reserva: pendiente, confirmada, cancelada, vencida. En lugar de usar simples cadenas de texto o condicionales dispersos, este objeto permite centralizar la lógica de transición de estados, validar acciones según el estado actual y facilitar el mantenimiento del sistema.
- **PolíticaDeCancelación** puede encapsular las reglas que determinan si una reserva puede cancelarse, si hay penalización, si depende del tipo de habitación o del tiempo restante. Este objeto permite separar la lógica de negocio del flujo operativo, y puede incluso variar según el contexto (por ejemplo, diferentes políticas para clientes frecuentes).

Incluir este tipo de objetos tiene múltiples beneficios:

- Encapsula la lógica: evita que las reglas estén dispersas en múltiples clases o métodos.
- Facilita el cambio: si la política cambia, solo se modifica el objeto correspondiente.
- Promueve la reutilización: el mismo objeto puede aplicarse en distintos contextos.
- Mejora la claridad conceptual: el modelo refleja no solo lo que el sistema hace, sino cómo decide hacerlo.

Desde la perspectiva de Sommerville (2005), esto se vincula con el principio de abstracción: modelar lo esencial del sistema, incluyendo sus reglas y decisiones, no sólo sus datos.

Y desde Larman (2005), responde al enfoque de responsabilidad distribuida, donde cada objeto tiene una función clara y autónoma.

En síntesis, incluir al menos un objeto conceptual o abstracto permite que el modelo no sea solo una representación estructural, sino también una expresión del comportamiento y la lógica del sistema, lo que lo vuelve más robusto, coherente y alineado con los principios del paradigma orientado a objetos.

Recomendaciones al Momento de Identificar Objetos de un Escenario

- No todos los sustantivos deben convertirse en clases: algunos pueden ser atributos o valores.
- Evitar clases técnicas como “BaseDeDatos” o “Formulario” si no aportan al modelo conceptual.
- Incluir al menos un objeto **conceptual o abstracto** que represente lógica del sistema (ej. EstadoReserva, PolíticaDeCancelación).
- Usar la simulación grupal como fuente rica para observar roles, mensajes y estados.
- Validar los objetos identificados con los **requisitos funcionales** del sistema.

UML como Herramienta de Modelado en el Paradigma Orientado a Objetos

En el desarrollo de sistemas orientados a objetos, el modelado conceptual cumple un rol fundamental: permite representar las entidades, relaciones y comportamientos que conforman el sistema antes de su implementación técnica.

Para llevar a cabo este modelado de manera rigurosa, comunicable y estandarizada, se utiliza UML (Unified Modeling Language), un lenguaje visual que se ha convertido en el estándar internacional para el modelado de software.

UML no es una metodología, sino un conjunto de notaciones gráficas que permiten describir distintos aspectos de un sistema: desde su estructura estática (clases, objetos, relaciones) hasta su comportamiento dinámico (interacciones, estados, actividades).

Su principal fortaleza radica en que facilita la comprensión compartida entre analistas, diseñadores, programadores y usuarios, al ofrecer una representación clara y formal de los componentes del sistema.

En el contexto del paradigma orientado a objetos, UML permite plasmar los principios de encapsulamiento, abstracción, herencia y polimorfismo en diagramas que reflejan cómo se organizan y colaboran los objetos.

Además, favorece la reflexión sobre la distribución de responsabilidades, la evolución del sistema y la validación de requisitos funcionales.

Vistas en UML: Una Mirada Integral al Sistema Orientado a Objetos

UML (Unified Modeling Language) organiza el modelado de sistemas en distintas vistas complementarias, cada una enfocada en un aspecto específico del sistema.

Esta división permite representar desde los requisitos funcionales hasta la arquitectura técnica, pasando por el modelado lógico y el comportamiento dinámico.

Según Sommerville (2005), esta multiplicidad de vistas responde a la necesidad de abordar el sistema desde diferentes niveles de abstracción, facilitando la comunicación entre perfiles diversos (analistas, diseñadores, desarrolladores, usuarios).

1. Vista Estructural (estática)

Propósito: Representar la organización interna del sistema, es decir, cómo se estructuran sus componentes, qué atributos poseen y cómo se relacionan entre sí.

Características:

- Se enfoca en las **clases, objetos, componentes y nodos** que conforman el sistema.
- Permite visualizar la **persistencia de datos**, la **jerarquía de herencia**, y la **composición de objetos**.
- Es clave para definir la arquitectura lógica del sistema.

Diagramas incluidos:

- **Diagrama de clases:** muestra las clases, sus atributos, métodos y relaciones (asociación, herencia, composición, etc.).
- **Diagrama de objetos:** representa instancias concretas de clases en un momento determinado.
- **Diagrama de componentes:** describe los módulos físicos del sistema (bibliotecas, servicios, etc.).
- **Diagrama de despliegue:** representa cómo se distribuyen los componentes en la infraestructura física (servidores, nodos, dispositivos).

Ejemplo aplicado: En un sistema de reservas, esta vista incluiría clases como Cliente, Reserva, Habitación, y sus relaciones (ej. un cliente puede tener varias reservas).

2. Vista de Comportamiento (dinámica)

Propósito: Describir cómo se comporta el sistema en respuesta a eventos, cómo se ejecutan los procesos y cómo cambian los estados de los objetos.

Características:

- Se centra en la **interacción entre objetos**, el **flujo de control**, y la **evolución de estados**.
- Permite modelar **escenarios de uso**, **procesos internos**, y **reacciones ante estímulos externos**.
- Es fundamental para validar la lógica funcional del sistema.

Diagramas incluidos:

- **Diagrama de casos de uso:** representa las funcionalidades del sistema desde la perspectiva de los actores externos.
- **Diagrama de actividades:** modela flujos de trabajo, decisiones, concurrencia y ciclos.
- **Diagrama de estados:** muestra los distintos estados de un objeto y las transiciones entre ellos.
- **Diagrama de secuencia:** representa la interacción temporal entre objetos en un escenario específico.
- **Diagrama de comunicación:** muestra cómo los objetos colaboran entre sí, destacando los vínculos más que el tiempo.

Ejemplo aplicado: En el proceso de “confirmar una reserva”, esta vista mostraría cómo el Cliente interactúa con el GestorDeReservas, cómo se valida la disponibilidad, y cómo se actualiza el estado de la Reserva.

3. Vista Funcional (requisitos)

Propósito: Capturar qué debe hacer el sistema, sin entrar en detalles de implementación o aspectos técnicos.

Características:

- Se enfoca en los **requisitos funcionales** del sistema.
- Permite identificar las **funcionalidades clave**, los **actores involucrados**, y las **interacciones esperadas**.
- Es especialmente útil en las primeras etapas del análisis.

Diagramas incluidos:

- **Diagrama de casos de uso:** describe las acciones que los usuarios pueden realizar y cómo el sistema responde.

Ejemplo aplicado: Un caso de uso como “Realizar reserva” incluiría al actor Cliente, el sistema como límite, y posibles extensiones como “Enviar confirmación” o “Solicitar pago”.

4. Vista Lógica del Diseño

Propósito: Mostrar cómo se organiza el sistema desde una perspectiva de diseño orientado a objetos, incluyendo patrones, responsabilidades y colaboraciones.

Características:

- Se enfoca en la **distribución de responsabilidades** entre objetos.
- Permite aplicar principios como **bajo acoplamiento**, **alta cohesión**, y **encapsulamiento**.
- Es clave para construir un diseño flexible, mantenible y escalable.

Diagramas incluidos:

- **Diagrama de clases**
- **Diagramas de interacción** (secuencia y comunicación)
- **Diagramas de colaboración estructurada** (UML 2.x)

Ejemplo aplicado: En el diseño de una reserva, esta vista definiría qué clase se encarga de validar disponibilidad, cuál gestiona el estado, y cómo se delegan responsabilidades entre objetos.

5. Vista Física y de Implementación

Propósito: Representar cómo se implementa y despliega el sistema en el entorno real, incluyendo aspectos técnicos y de infraestructura.

Características:

- Se enfoca en los **componentes físicos**, la **infraestructura de ejecución**, y las **dependencias técnicas**.
- Es útil en etapas avanzadas del desarrollo, cuando se define la arquitectura de software y hardware.

Diagramas incluidos:

- **Diagrama de componentes:** muestra los módulos de software y sus interfaces.
- **Diagrama de despliegue:** representa la distribución de componentes en servidores, dispositivos o nodos.

Ejemplo aplicado: En un sistema web, esta vista mostraría cómo el módulo de reservas se despliega en un servidor de aplicaciones, cómo se conecta con la base de datos, y qué servicios externos utiliza.

Tabla comparativa de vistas en UML

Vista UML	Propósito	Características principales	Diagramas incluidos
Vista estructural (estática)	Representar la organización interna del sistema.	- Define clases, atributos, relaciones. - Muestra la arquitectura lógica. - Enfatiza persistencia y estructura.	- Diagrama de clases - Diagrama de objetos - Diagrama de componentes - Diagrama de despliegue
Vista de comportamiento (dinámica)	Describir cómo se comporta el sistema ante eventos.	- Modela interacciones, procesos y cambios de estado. - Representa lógica funcional. - Enfatiza el flujo de control.	- Diagrama de casos de uso - Diagrama de actividades - Diagrama de estados - Diagrama de secuencia - Diagrama de comunicación
Vista funcional (requisitos)	Capturar qué debe hacer el sistema desde la perspectiva del usuario.	- Enfocada en funcionalidades clave. - Útil en etapas iniciales. - Relaciona actores y acciones.	Diagrama de casos de uso
Vista lógica del diseño	Mostrar cómo se distribuyen responsabilidades entre objetos.	- Aplica principios POO (encapsulamiento, cohesión, acoplamiento). - Enfatiza el diseño conceptual.	- Diagrama de clases - Diagramas de interacción - Diagramas de colaboración estructurada
Vista física y de implementación	Representar cómo se despliega el sistema en el entorno técnico real.	- Define arquitectura técnica. - Muestra infraestructura y dependencias. - Útil en etapas avanzadas.	- Diagrama de componentes - Diagrama de despliegue

Cada vista responde a una pregunta distinta:

- ¿Qué hace el sistema? → Vista funcional
- ¿Cómo está organizado? → Vista estructural
- ¿Cómo se comporta? → Vista de comportamiento
- ¿Cómo se diseña? → Vista lógica
- ¿Dónde se ejecuta? → Vista física

Incluir múltiples vistas permite construir un modelo **completo, coherente y comunicable**, alineado con los principios del paradigma orientado a objetos y con las buenas prácticas de ingeniería de software.

Como parte del desarrollo conceptual del TP 02, se abordarán tres tipos de diagramas UML fundamentales para el modelado de sistemas orientados a objetos: el **diagrama de casos de uso**, el **diagrama de clases** y el **diagrama de secuencia**. Cada uno de ellos será explicado en detalle, con ejemplos y criterios de aplicación, para facilitar su comprensión y correcta implementación en las actividades solicitadas. Estos diagramas ofrecen perspectivas complementarias sobre el sistema: el diagrama de casos de uso permite identificar las funcionalidades desde el punto de vista del usuario; el diagrama de clases representa la estructura interna del sistema, sus entidades y relaciones; y el diagrama de secuencia muestra cómo los objetos colaboran en un escenario específico para cumplir una funcionalidad. El dominio de estos modelos no solo permitirá completar el TP 02 con solidez conceptual, sino también construir representaciones transferibles, justificadas y alineadas con los principios del paradigma orientado a objetos.

Diagrama de Caso de Uso – Representación Funcional del Sistema

El diagrama de caso de uso es el punto de partida en el modelado de sistemas orientados a objetos, ya que permite representar las **funcionalidades principales del sistema desde la perspectiva del usuario o actor externo**. Su propósito no es mostrar cómo se implementa el sistema, sino qué servicios ofrece y quiénes los utilizan. En este sentido, constituye una herramienta clave para **capturar los requisitos funcionales**, facilitar la comunicación con los usuarios y delimitar el alcance del sistema.

Cada caso de uso representa una **unidad funcional significativa**, como “Realizar reserva”, “Consultar disponibilidad” o “Cancelar operación”, y se vincula con uno o más actores que interactúan con el sistema. A través de este modelo, es posible visualizar el sistema como un conjunto de servicios organizados, lo que permite avanzar luego hacia el modelado estructural y dinámico con mayor claridad.

Además, el diagrama de caso de uso permite establecer **relaciones entre casos** (como inclusión o extensión), definir los **límites del sistema**, y detectar **escenarios alternativos o excepcionales**. Su construcción no requiere conocimientos técnicos avanzados, pero sí una comprensión profunda del dominio del problema, lo que lo convierte en una excelente herramienta para iniciar el proceso de análisis y diseño.

A continuación, se presentarán los elementos que componen el diagrama de caso de uso, su notación, ejemplos aplicados y criterios para su elaboración, con el fin de facilitar su correcta interpretación y aplicación en el modelado de sistemas.

Elementos del Diagrama de Caso de Uso

Actor

- **Definición:** Representa una entidad externa que interactúa con el sistema. Puede ser una persona, otro sistema o un dispositivo.
- **Función:** Inicia o participa en uno o más casos de uso.
- **Notación:** Figura de persona (stickman) con el nombre debajo.
- **Ejemplos:** Cliente, Empleado, Administrador, Sistema de pagos.

Nota: Un actor no necesariamente corresponde a una clase en el sistema. Representa un rol funcional, no una implementación técnica.

Caso de Uso

- **Definición:** Representa una funcionalidad específica que el sistema ofrece a uno o más actores.
- **Función:** Describe una unidad de comportamiento observable desde el exterior.
- **Notación:** Óvalo con el nombre del caso de uso dentro.
- **Ejemplos:** Realizar reserva, Consultar disponibilidad, Cancelar operación.

Recomendación: Usar verbos en infinitivo que expresen acciones concretas.

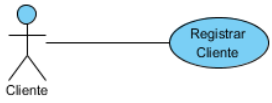
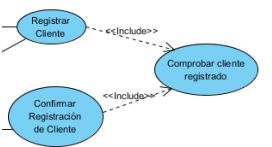
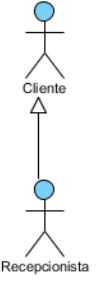
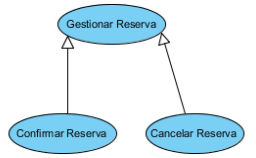
Sistema (límites)

- **Definición:** Delimita el alcance del sistema modelado.
- **Función:** Distingue lo que está dentro del sistema (casos de uso) de lo que está fuera (actores).
- **Notación:** Rectángulo que contiene los casos de uso, con el nombre del sistema en la parte superior.

Importancia: Ayuda a definir qué funcionalidades pertenecen al sistema y cuáles son externas o delegadas.

Relaciones

UML permite representar distintos tipos de vínculos entre actores y casos de uso, así como entre los propios casos.

Relación UML	Función principal	Notación	Ejemplo típico	
Asociación	Vincula un actor con un caso de uso. Representa participación o interacción.	Línea continua sin dirección (no tiene flecha)	Cliente participa en Realizar reserva	
Inclusión	Un caso de uso siempre incluye otro. Reutiliza funcionalidad común.	Línea discontinua con una flecha abierta apuntando al caso de uso que se incluye con el estereotipo <<include>>	Realizar reserva incluye Validar disponibilidad	
Extensión	Un caso de uso puede extenderse a otro bajo ciertas condiciones. Modela comportamiento opcional.	Línea discontinua con una flecha abierta apuntando al caso de uso que del que se extiende con el estereotipo <<extend>>	Realizar reserva puede extender Solicitar pago anticipado	
Generalización (entre actores)	Un actor hereda el comportamiento de otro. Representa especialización de roles.	Flecha con punta cerrada apuntando a la generalización.	Empleado generaliza Recepcionista y Administrador	
Generalización (entre casos de uso)	Un caso de uso hereda el comportamiento de otro. Representa variantes funcionales.	Flecha con punta abierta	Gestionar reserva generaliza Confirmar y Cancelar	

Estrategias para Identificar Casos de Uso en un Escenario

1. Leer el escenario desde la perspectiva del usuario

- Preguntarse: *¿Qué espera lograr el usuario al interactuar con el sistema?*
- Identificar las **acciones concretas** que el usuario realiza o solicita.
- Separar lo que hace el sistema internamente de lo que el usuario percibe como funcionalidad.

Ejemplo: En el escenario “Un cliente accede al sistema para reservar una habitación”, el caso de uso sería Realizar reserva, no Actualizar base de datos.

2. Detectar verbos en infinitivo

- Buscar verbos que expresen **intenciones o tareas** del usuario: *consultar, registrarse, modificar, cancelar, pagar*.
- Evitar verbos técnicos o de bajo nivel como *guardar, procesar, enviar datos*, que suelen pertenecer a la implementación.

Consejo: Si el verbo responde a una necesidad del usuario, probablemente sea un caso de uso.

3. Identificar los actores involucrados

- Reconocer los **roles funcionales** que interactúan con el sistema: personas, sistemas externos, dispositivos.
- Para cada actor, preguntarse: *¿Qué funcionalidades necesita? ¿Qué acciones puede realizar?*

Ejemplo: Un Administrador puede tener casos de uso como Gestionar usuarios, Consultar reportes, Modificar políticas.

4. Analizar los objetivos del sistema

- Revisar los **requisitos funcionales** y los objetivos del sistema.
- Traducir cada objetivo en una o más funcionalidades observables.

Ejemplo: Si el sistema busca “automatizar el proceso de reservas”, los casos de uso podrían incluir Realizar reserva, Confirmar disponibilidad, Enviar notificación.

5. Identificar eventos que inician procesos

- Detectar **eventos externos** que desencadenan una respuesta del sistema.
- Cada evento relevante puede corresponder a un caso de uso.

Ejemplo: “El cliente solicita cancelar una reserva” → caso de uso: Cancelar reserva.

6. Validar con preguntas clave

- ¿Este caso de uso aporta valor al actor?
- ¿Tiene un inicio claro y un resultado observable?
- ¿Puede ejecutarse de forma independiente?
- ¿Está alineado con los objetivos del sistema?

Si la respuesta es afirmativa, probablemente sea un caso de uso válido.

Como resultado del proceso de identificación de casos de uso, y como forma de documentar de manera clara y estructurada las funcionalidades del sistema, se presenta la siguiente tabla.

Actor	Caso de uso (Verbo + Sustantivos)	Descripción funcional
Cliente	Realizar reserva	El cliente solicita una habitación para una fecha determinada.
Cliente	Consultar disponibilidad	El cliente verifica si hay habitaciones disponibles según sus criterios.
Cliente	Cancelar reserva	El cliente anula una reserva previamente confirmada.
Cliente	Registrarse en sistema	El cliente crea una cuenta para poder operar en el sistema.
Recepcionista	Confirmar reserva	El empleado valida y aprueba una reserva realizada por el cliente.
Administrador	Modificar política de cancelación	El administrador actualiza las condiciones bajo las cuales se puede cancelar una reserva.
Recepcionista	Verificar registro de cliente	El recepcionista comprueba si el cliente está registrado antes de permitir otras acciones.
Recepcionista	Validar disponibilidad	El recepcionista analiza si hay habitaciones libres para las fechas solicitadas.
Sistema	Enviar notificación	El sistema comunica al cliente el estado de su reserva.
Administrador	Consultar reportes	El administrador accede a informes sobre reservas, ocupación y estadísticas.
Administrador	Crear habitación	El administrador crea una nueva habitación para que quede disponible para su alquiler o reserva.

Es importante tener en cuenta que, al identificar los casos de uso de un sistema, no deben considerarse únicamente las funcionalidades visibles para el usuario final, sino también aquellas que permiten configurar, mantener y alimentar el sistema para que esas funcionalidades sean posibles.

Por ejemplo, en un sistema de reservas, suele detectarse rápidamente el caso de uso “Gestionar reserva”, pero se omiten otros casos esenciales como “Crear habitación”, “Definir tipo de habitación” o “Actualizar disponibilidad”, que son necesarios para que el sistema funcione correctamente. Ignorar estos casos puede llevar a modelos incompletos o inconsistentes.

Por eso, se recomienda analizar también los procesos de administración, configuración y mantenimiento del sistema, y considerar a los actores internos que los ejecutan.

Consideración sobre el Actor “Sistema”

En algunos modelos, especialmente en etapas formativas o de documentación funcional, se incluye al **Sistema** como actor para representar aquellas acciones que se ejecutan de forma **automática, programada o interna**, sin intervención directa de una persona física. Aunque en UML los actores suelen ser entidades externas al sistema, esta decisión de modelado permite

- **Visualizar responsabilidades funcionales** que el sistema asume como respuesta a eventos o condiciones.
- **Diferenciar claramente** entre lo que el usuario solicita y lo que el sistema realiza como parte del flujo funcional.

Incluir al “Sistema” como actor en el diagrama de casos de uso permite representar acciones automatizadas que, aunque no son iniciadas por actores humanos, forman parte esencial del comportamiento observable del sistema. Esta decisión de modelado contribuye a mantener la coherencia funcional del diagrama y fortalece la trazabilidad entre los casos de uso definidos y los requisitos funcionales establecidos.

Del Análisis al Modelado: Construcción del Diagrama de Casos de Uso

Una vez identificados y organizados los casos de uso a partir del análisis funcional del escenario, el siguiente paso consiste en representarlos gráficamente utilizando una herramienta CASE. Esta instancia permite traducir las acciones detectadas en un modelo visual conforme a la notación UML, incorporando actores, casos de uso, relaciones y límites del sistema. La representación diagramática no solo facilita la validación del modelo, sino que también constituye un soporte técnico para las etapas posteriores de diseño y desarrollo.

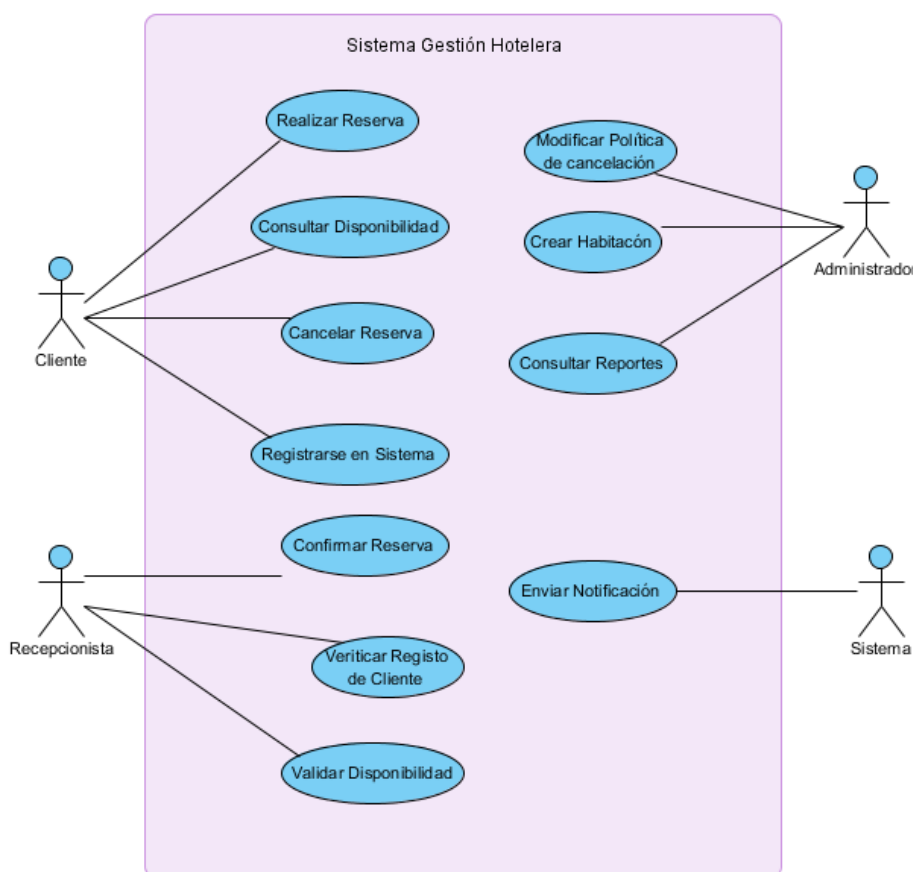


Fig. 1. Modelo parcial del sistema de gestión hotelera

El Caso de Uso como Modelo Textual Descriptivo

El caso de uso es una unidad funcional que representa cómo el sistema responde a una interacción iniciada por un actor. Más allá de su representación gráfica en el diagrama, el caso de uso puede ser desarrollado como un **modelo textual**, que permite describir con mayor profundidad qué hace el sistema y cómo lo hace.

Este modelo incluye elementos como:

- Objetivo del actor
- Condiciones previas y posteriores
- Flujos de eventos principales y alternativos
- Reglas de negocio
- Relaciones con otros casos de uso

Según el propósito del análisis, este modelo puede adoptar distintos niveles de detalle:

Tipo de modelo	Propósito	Cuándo se utiliza
Modelo esencial	Describe la intención del usuario sin referencia a tecnología ni interfaz.	En etapas tempranas del análisis, para validar requisitos con actores no técnicos.
Modelo real	Describe cómo se implementa la funcionalidad, incluyendo interfaz y restricciones.	En etapas de diseño o documentación técnica, útil para desarrolladores y testers.

Tipo de modelo	Propósito	Cuándo se utiliza
Caso de uso de alto nivel	Ofrece una descripción general de una funcionalidad del sistema, redactada de forma sintética, sin detallar flujos internos, condiciones ni variantes.	En etapas iniciales del análisis, para identificar bloques funcionales, delimitar el alcance del sistema y facilitar la comunicación con actores no técnicos.
Caso de uso extendido	Incorpora mayor nivel de detalle sobre un caso de uso base, incluyendo alternativas, secuencias típicas y variantes condicionales del flujo de eventos.	En etapas posteriores del análisis, cuando se requiere representar comportamientos opcionales, excepciones o ramificaciones lógicas. Será abordado en guías específicas más adelante.

Este enfoque permite adaptar el nivel de abstracción según el objetivo del proyecto, manteniendo la coherencia entre lo que se modela, lo que se implementa y lo que se valida.

Para el Trabajo Práctico 02 se trabajará con casos de uso esenciales y de alto nivel, con el objetivo de identificar y describir las funcionalidades principales del sistema desde una perspectiva conceptual y funcional.

Estos modelos permitirán representar las intenciones de los actores y las áreas funcionales del sistema sin entrar en detalles técnicos ni en flujos alternativos.

El caso de uso extendido, que incorpora variantes condicionales y secuencias alternativas de eventos, será abordado en trabajos posteriores.

A continuación, se presenta el formato propuesto para el caso de uso de alto nivel, seguido de un ejemplo aplicado al caso de uso *Realizar reserva*.

Nombre del caso de uso: *(Verbo + Sustantivo, por ejemplo: Realizar reserva)*

Actor principal: *(Quién inicia la acción: Cliente, Administrador, etc.)*

Precondición: *(Estado necesario antes de iniciar el caso de uso)*

Intención del actor: *(Qué desea lograr el actor al interactuar con el sistema)*

Propósito del sistema: *(Qué funcionalidad ofrece el sistema para satisfacer esa intención)*

Descripción general: *(Redacción sintética que explique qué hace el sistema en relación con esta funcionalidad, sin detallar flujos ni condiciones)*

Postcondición: *(Estado del sistema una vez completada la acción)*

Ejemplo

Nombre del caso de uso: Realizar reserva

Actor principal: Cliente

Precondición:

- El cliente debe estar registrado.
- Debe haber disponibilidad de habitaciones para las fechas solicitadas.

Intención del actor: Reservar una habitación para una fecha determinada, según sus preferencias.

Propósito del sistema: Permitir al cliente seleccionar fechas, tipo de habitación y registrar la reserva en el sistema.

Descripción general: El cliente accede al sistema para reservar una habitación, indicando fechas, tipo de habitación y preferencias. El sistema registra la solicitud y confirma la operación si hay disponibilidad.

Postcondición:

- Se genera una reserva con estado “pendiente” o “confirmada”.
- El cliente recibe una confirmación de la operación.

Introducción al Modelo de Clases

El **modelo de clases** describe los elementos estructurales del sistema desde una perspectiva orientada a objetos. A diferencia del diagrama de casos de uso, que se enfoca en las funcionalidades y las interacciones externas, el modelo de clases permite identificar **entidades internas**, sus **propiedades** y **comportamientos**, así como las **relaciones** que existen entre ellas.

Este modelo responde a preguntas como:

- ¿Qué conceptos maneja el sistema?
- ¿Qué información debe almacenar cada entidad?
- ¿Qué operaciones puede realizar cada clase?
- ¿Cómo se vinculan las clases entre sí?

El modelo de clases se construye a partir de los casos de uso definidos previamente, extrayendo los **objetos relevantes**, sus **atributos funcionales** y las **acciones que deben soportar**.

La Clase como Conceptualización Técnica del Objeto Identificado

En el marco del análisis orientado a objetos, la **clase** representa la **conceptualización técnica** de un objeto o entidad relevante para el sistema. Esta conceptualización surge del análisis funcional, especialmente de los escenarios narrados y los casos de uso definidos previamente. Es decir, **cada clase modela un concepto que fue identificado como necesario para que el sistema cumpla con sus funcionalidades**.

Por ejemplo, si en el caso de uso *Realizar reserva* se menciona que el cliente selecciona una habitación, se registra una fecha y se genera una confirmación, entonces las entidades *Reserva*, *Cliente* y *Habitación* deben ser modeladas como clases, cada una con sus atributos y operaciones correspondientes.

Relación con los Principios de la Orientación a Objetos

La clase no solo organiza información: también incorpora los principios fundamentales del paradigma orientado a objetos:

- **Abstracción:** La clase permite representar los aspectos relevantes de una entidad del mundo real, omitiendo detalles innecesarios. Se enfoca en lo que el sistema necesita saber y hacer con ese objeto, no en cómo está implementado internamente.
- **Encapsulamiento:** La clase agrupa en una misma unidad los datos (atributos) y los comportamientos (operaciones) que le corresponden. Esto permite mantener la coherencia interna y controlar cómo se accede o modifica la información.
- **Ocultamiento de información:** A través de la definición de visibilidad (por ejemplo, atributos privados y métodos públicos), la clase protege sus datos internos y expone solo lo necesario para interactuar con otras partes del sistema. Esto mejora la seguridad, la modularidad y la mantenibilidad del software.

Conclusión: La clase es el resultado de **traducir conceptualmente los objetos del escenario funcional** en estructuras técnicas que el sistema podrá implementar. No se trata solo de dibujar rectángulos con atributos y métodos, sino de **modelar con precisión y coherencia** aquello que el sistema necesita representar, controlar y relacionar para cumplir con los casos de uso definidos.

¿Qué es una Clase?

Una **clase** representa un concepto del sistema que agrupa de forma coherente los **atributos** (información que debe almacenar) y las **operaciones** (acciones que puede realizar o recibir). Es la unidad básica del modelo estructural y surge como **conceptualización técnica** de los objetos o entidades identificados en los escenarios funcionales a través de los casos de uso.

Estructura de una Clase en UML

En el diagrama, cada clase se representa como un **rectángulo dividido en tres secciones horizontales**:

Sección	Contenido	Ejemplo
Nombre de la clase	Identifica el concepto modelado. Se escribe centrado y en negrita.	Reserva
Atributos	Propiedades o datos que la clase debe almacenar. Se escriben en formato nombre: tipo.	fechaInicio: Date
Operaciones	Acciones que la clase puede realizar o recibir. Se escriben en formato nombre(parámetros): tipoRetorno.	crearReserva(fechaInicio, fechaFin): Boolean

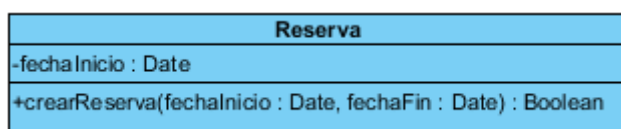


Fig.2 Representación de una clase en UML

Visibilidad

Cada atributo u operación puede incluir un símbolo de **visibilidad**, que indica cómo se accede a ese elemento desde otras clases:

- + Público: accesible desde cualquier clase
- - Privado: accesible solo dentro de la clase
- # Protegido: accesible desde la clase y sus subclases

Ejemplo: -estado: String indica que el atributo estado es privado.

Ahora que ya definimos la estructura de la clase, podemos avanzar con las **asociaciones**, que permiten representar cómo se vinculan las clases entre sí dentro del sistema. Este paso es clave para construir un **modelo coherente y funcional**, ya que las asociaciones reflejan las dependencias, colaboraciones y estructuras internas que emergen de los casos de uso.

¿Qué es una Asociación?

Una **asociación** en UML representa una **relación estructural** entre dos clases. Indica que existe una conexión lógica o funcional entre las instancias de esas clases, y puede incluir información sobre **multiplicidad**, **navegabilidad**, **roles** y **tipo de vínculo**.

Elementos de una Asociación

Elemento	Descripción	Ejemplo aplicado
Línea de asociación	Une dos clases que se relacionan. Puede ser bidireccional o unidireccional.	Cliente ----- Reserva
Multiplicidad	Indica cuántas instancias de una clase pueden estar relacionadas con otra.	1 a 0..* (un cliente puede tener muchas reservas)
Rol	Nombre que describe el papel que cada clase cumple en la relación.	cliente / reservas
Navegabilidad	Flecha que indica si una clase puede acceder a la otra.	Reserva → Cliente
Etiqueta opcional	Puede incluir el nombre de la relación si es relevante.	realiza

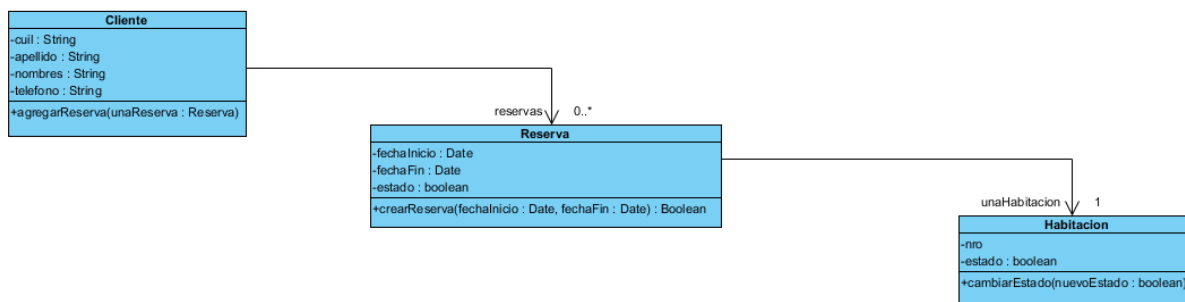


Fig. 3. Diagrama de clase

Tipos Especiales de Asociación

Además de la asociación simple, UML permite representar relaciones más específicas:

Tipo	Símbolo	Descripción
Agregación	Rombo vacío 	Relación “todo-parte” donde las partes pueden existir independientemente.
Composición	Rombo negro 	Relación “todo-parte” donde las partes dependen del todo para existir.
Generalización	Flecha cerrada vacía. 	Relación de herencia: una clase hija hereda atributos y operaciones de la clase padre.

Ejemplo de composición: Una **Reserva** puede tener una **Confirmación**, pero si se elimina la reserva, la confirmación también desaparece.

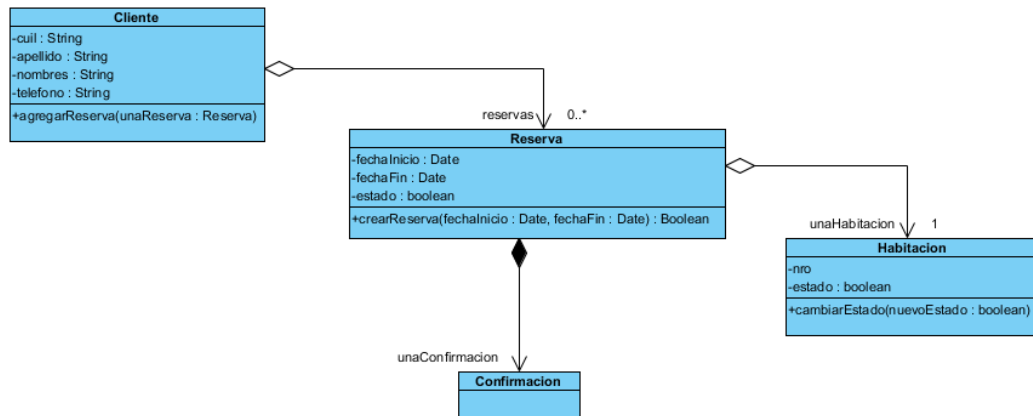


Fig. 4. Diagrama de clase con algunas asociaciones especiales

Generalización y Herencia en el Modelo de Clases

La **generalización** en UML representa una relación jerárquica entre clases, donde una clase más específica (subclase) hereda atributos y operaciones de una clase más general (superclase). Esta relación se indica gráficamente mediante una línea con un **triángulo vacío apuntando hacia la clase general**. Desde el punto de vista conceptual, la generalización permite **reutilizar comportamientos comunes** y **organizar el modelo de forma más abstracta y coherente**.

Relación con el Principio de Herencia

La generalización se fundamenta en el principio de **herencia**, uno de los pilares de la programación orientada a objetos. Este principio establece que una clase puede **heredar** las características (atributos y métodos) de otra, evitando duplicación y facilitando la extensibilidad del sistema.

- **Herencia simple:** una subclase hereda de una única superclase.
- **Herencia múltiple** (no representada directamente en UML): una clase hereda de varias superclases (según el lenguaje de implementación).

Nota: Utilizar generalización permite construir modelos más limpios, reutilizables y alineados con los principios del paradigma orientado a objetos. En el TP, se recomienda aplicar este recurso cuando se identifiquen entidades que comparten comportamientos y propiedades, pero cumplen roles distintos en el sistema.

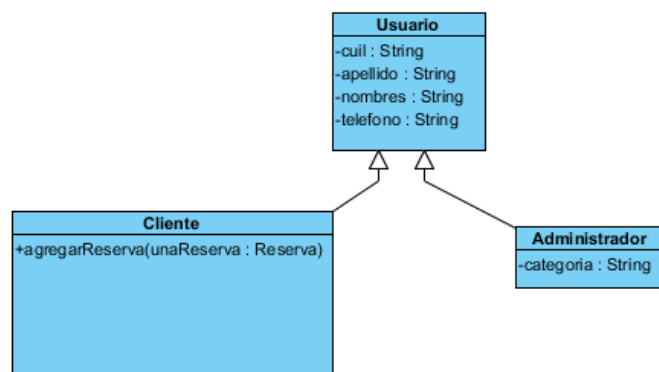


Fig. 5. Representación de una Generalización

Diagrama de Secuencia

El **Lenguaje Unificado de Modelado (UML)** es un estándar ampliamente utilizado en el modelado y documentación de sistemas de software. Su objetivo principal es proporcionar una representación visual, clara y estructurada de la interacción entre los componentes de un sistema. Dentro de UML, los **diagramas de secuencia** desempeñan un papel crucial a la hora de describir el flujo de mensajes entre objetos en un escenario particular.

El diagrama de secuencia es uno de los modelos dinámicos de UML, es decir, muestra la interacción entre los objetos mediante mensajes en el tiempo. Son especialmente útiles para entender el comportamiento de un sistema en diferentes situaciones, ya que permiten visualizar el orden de las interacciones y la cronología de los eventos.

Este apunte tiene como objetivo introducir algunos de los elementos más utilizados en los diagramas de secuencia de UML, tales como participantes, mensajes, constraints y fragmentos combinados, brindando ejemplos prácticos para su comprensión. Sin embargo, es importante destacar que no se trata de una presentación exhaustiva de todas las posibilidades que ofrece UML en este tipo de diagramas.

Elementos Básicos

- **Instancias y actores:** representan entidades que interactúan en el sistema.
- **Mensajes:** la comunicación entre objetos (sincrónicos y asincrónicos).
- **Líneas de vida:** indican la existencia de un objeto en el tiempo.
- **Barra de activación:** indica que una instancia está activa (o instanciada) durante la interacción entre dos objetos.
- **Fragmentos:** condiciones y repeticiones en el flujo del sistema.

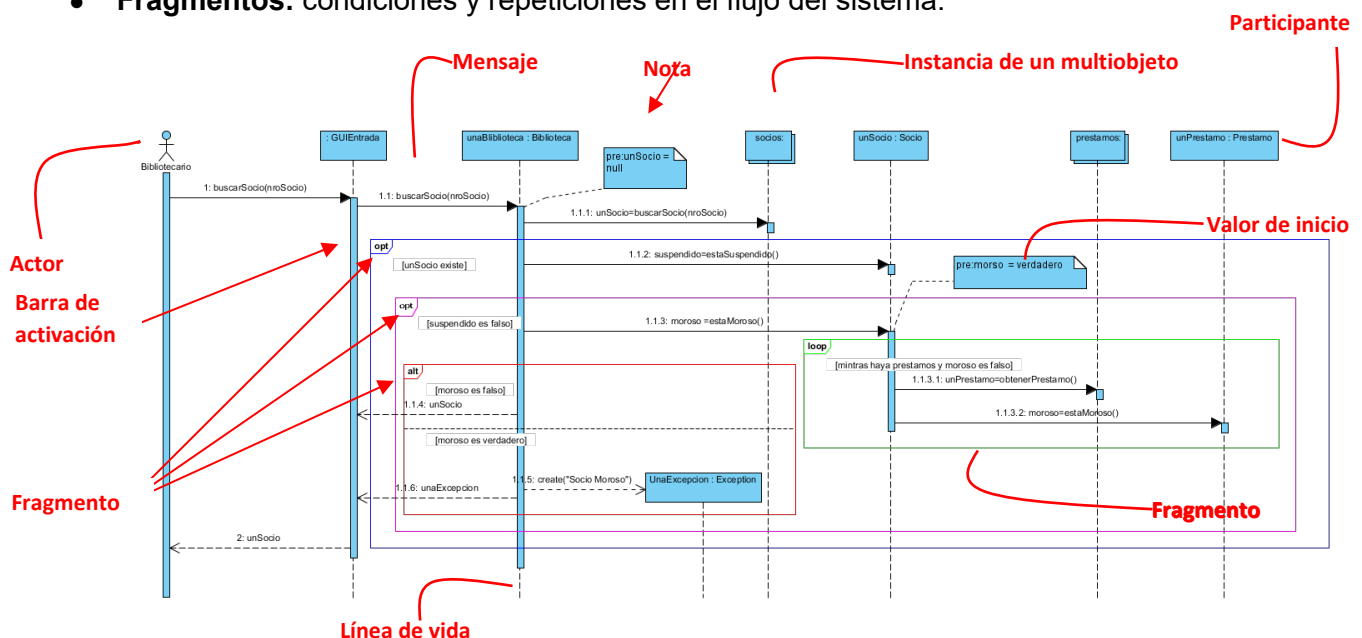


Fig. 6. Elementos del diagrama de secuencia

Actor

En el diagrama de secuencia el actor se representa mediante una Stick figure (figura de palo) conjunto a un sustantivo que representa el nombre del caso de uso.



Fig. 7. Actor

Participante

Es una entidad (instancia, clase, o un sistema) que envía o recibe mensajes durante la secuencia de eventos modelada. Se representa en la parte superior del diagrama como una caja con su nombre, de la cual descende una línea de vida. La instancia está identificada por un identificador y el nombre del concepto separados por dos puntos.

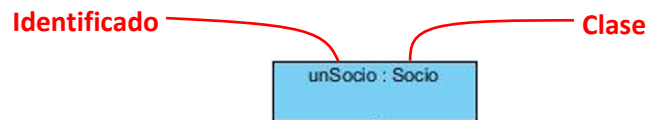


Fig. 8. Participante

Mensaje

- **Mensaje síncrono:** Un mensaje síncrono representa una llamada a una operación en la que el objeto que envía el mensaje espera una respuesta del objeto receptor antes de continuar su ejecución. Esta respuesta puede modelarse de dos formas:

A- mediante un mensaje de retorno, que muestra explícitamente que el receptor envía un valor o confirmación de vuelta al emisor;

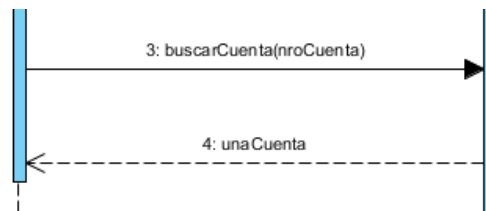


Fig. 9. Mensaje síncrono modelo A

B- integrando la idea de asignación directamente en el mensaje síncrono, por ejemplo: unSocio = buscarSocio(nroSocio), indicando que el emisor queda a la espera del resultado para continuar.

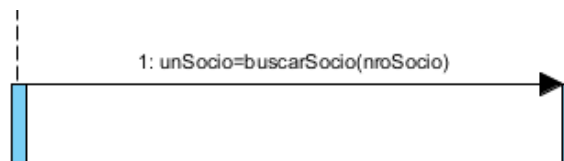


Fig. 10. Mensaje síncrono modelo A

- **Mensaje Asíncrono:** El emisor no espera una respuesta. Se representa con una línea continua y una flecha de punta abierta.

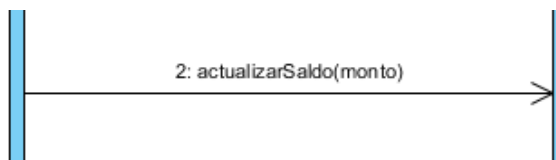


Fig. 11. Mensaje asíncrono

- **Mensaje de Creación:** Representa la creación de una nueva instancia durante la interacción entre instancias en el sistema y, a partir de él, se inicia la existencia de la instancia. y en general se representa con una línea continua y una flecha llena

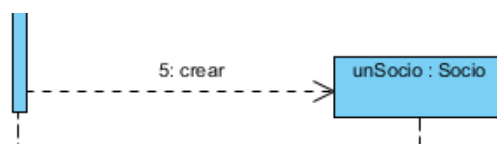


Fig. 12. Mensaje de creación de una instancia

Línea de Vida

Indica la existencia de una instancia o de un actor dentro del tiempo de ejecución del sistema. Pueden ser de los siguientes tipos:

- Persistente: en este caso, la instancia existe durante toda la ejecución del proceso y seguirá existiendo al finalizarlo.
- Temporal: son las instancias creadas y destruidas en una interacción.

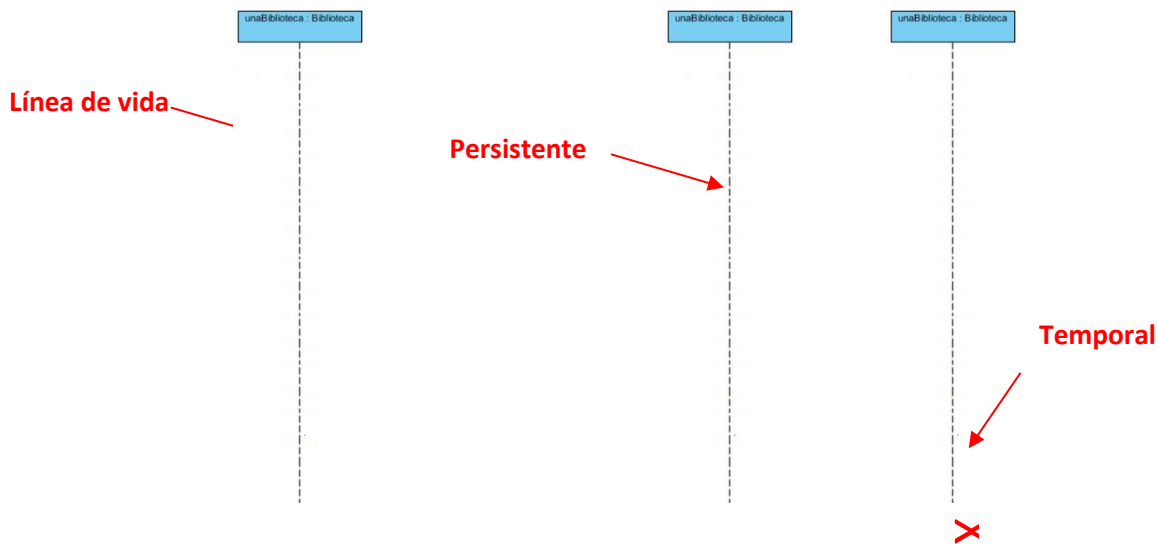


Fig. 13. Línea de vida de un participante

Barra de Activación

La barra de activación, también llamada rectángulo de activación, es un elemento visual de los diagramas de secuencia UML que representa el período durante el cual un objeto está ejecutando una acción o procesando un mensaje. Se dibuja como un rectángulo delgado superpuesto a la línea de vida del objeto, lo que nos permite ver cuánto tiempo dura la responsabilidad de la instancia.

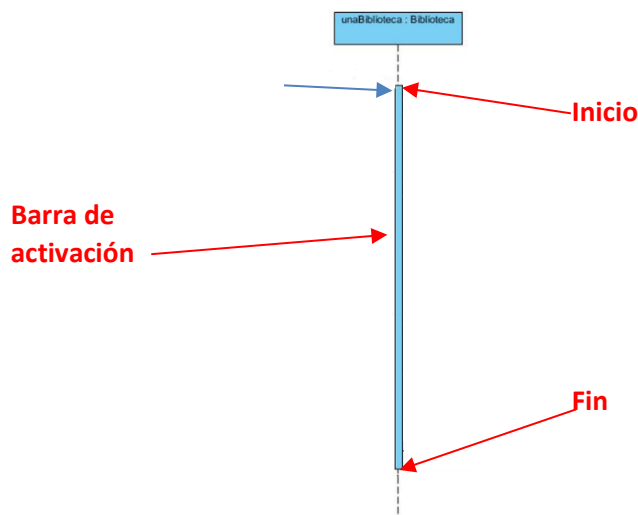


Fig.14. Barra de Activación

Fragmentos

Los fragmentos combinados nos permiten modelar situaciones que requieran representar flujos condicionales, bucles y otros comportamientos en las interacciones entre los objetos. Mediante estos fragmentos se pueden capturar escenarios más realistas en los que las interacciones no son solamente lineales, sino que también hay decisiones del tipo if/else, repeticiones o procesos concurrentes.

Se representa mediante un rectángulo que encierra todos los mensajes que forman el fragmento. En el rectángulo de la parte superior izquierda se debe indicar el tipo y, al lado, la condición que se debe evaluar, encerradas entre corchetes. La condición del fragmento no tiene por qué estar en un formato formal, por ejemplo: $[A > B]$, también puede ser una oración que deje bien clara cuál es la condición que se debe evaluar, por ejemplo: [mientras haya cliente y no se haya encontrado] para un fragmento del tipo loop. Otro ejemplo puede ser para un fragmento del tipo if/else: [si se encontró el cliente].

Algunos Tipos de Fragmentos

OPT y ALT

En el diagrama de secuencia para modelar un comportamiento condicional se utilizan fragmentos de tipo OPT o ALT. OPT se utiliza cuando los mensajes deben ejecutarse si se cumple la condición y, si no se cumple, se omiten. En cambio, ALT se utiliza cuando hay distintos caminos y solo se debe ejecutar uno de ellos, y cada uno de estos caminos tiene una condición propia.

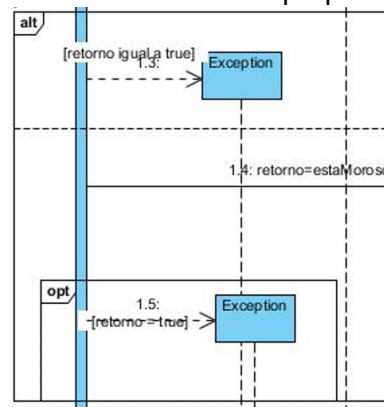


Fig.15. Fragmentos ALT y OPT

LOOP

El fragmento loop representa una secuencia de mensajes que se ejecutan repetidamente, es decir, una iteración, mientras se cumple una condición.

Las condiciones se pueden escribir utilizando lógica formal ($a < 100$) o un lenguaje natural (mientras haya clientes). El estándar de UML no restringe el formato, por lo que se puede adaptar al nivel de detalle requerido.

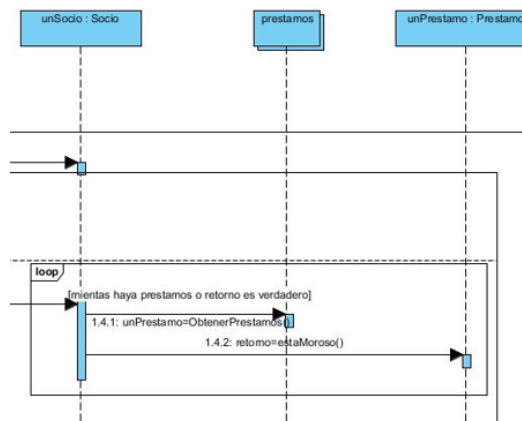


Fig.16. Fragmentos LOOP

Visibilidad en Diagramas de Secuencia según Craig Larman

En un diagrama de secuencia, para que se pueda establecer una interacción entre dos participantes debe existir una visibilidad entre ambos. La visibilidad determina cómo los objetos interactúan y acceden entre sí durante la ejecución de un caso de uso. Una correcta gestión de la visibilidad asegura una estructura bien encapsulada, con bajo acoplamiento y alta cohesión.

Tipos de Visibilidad Aplicados a Diagramas de Secuencia

Para poder establecer interacciones entre dos instancias en un diagrama de secuencias, debe existir alguna de las siguientes visibilidades entre ambas.

a) Visibilidad por Atributo (Almacenamiento Permanente)

En el **modelo de dominio** (diagrama de clases conceptual), las **asociaciones** representan relaciones significativas entre clases, las cuales suelen traducirse en **visibilidad por atributo** en el modelado detallado (diagrama de clases de diseño) y, consecuentemente, en interacciones en el **diagrama de secuencia**.

b) Visibilidad por Parámetro (Dependencia Temporal)

- Un objeto recibe una referencia **como parámetro de un método**.
- **Ejemplo en Diagrama de Secuencia:**
 - Un método procesarPago(MedioPago mp) permite que Pedido interactúe con MedioPago sin guardarlo como atributo.
- Cliente -> Pedido: procesarPago(medioPago)
- Pedido -> MedioPago: verificarSaldo()

c) Visibilidad por Creación Local

- Un objeto **crea otro** dentro de un método y lo usa.
- **Ejemplo en Diagrama de Secuencia:**
 - Factura crea un PDF temporal para generar un reporte.
- Factura -> PDF: new()
- PDF -> Factura: generarContenido()

d) Visibilidad Global (Evitar en el modelado OO)

- Acceso a objetos globales (ej: Singleton). **Larman lo desaconseja** por aumentar el acoplamiento.
- **Ejemplo (No recomendado):**
- Pedido -> InventarioGlobal: getStock()

Reglas Clave para Diagramas de Secuencia

- **Minimizar visibilidad global:** Preferir parámetros o atributos.
- **Mantener interacciones claras:** Si un objeto A envía muchos mensajes a B, quizá deban estar asociados (atributo).
- **Evitar crear objetos innecesarios:** Si un objeto solo se usa en un método, preferir visibilidad por parámetro.

Bibliografía

- Sommerville, I. (2005). *Ingeniería de software* (7.^a ed.). Prentice Hall. ISBN 84-7829-074-5.
- Larman, C. (2005). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Booch, G., Rumbaugh, J., & Jacobson, I. (1999). *The Unified Modeling Language User Guide*. Addison-Wesley.